

Shiv Nadar Institution of Eminence
Department of Computer Science and Engineering

Behavioral-Diff Detection of Malicious Browser Extension Updates

A sandboxed runtime analysis framework for supply-chain threat detection in Chromium extensions

Group Members: Bind Pratap Singh (2310110084), Suhani Deepak Agrawat (2310110717)
Programme: B.Tech CSE, Batch 2023–27
Project Advisor: Dr. Sweta Mishra
Date: May 23, 2026

Abstract

Browser extensions are a persistent supply-chain risk: a trusted, widely-installed extension can become malicious after a silent background update, reaching hundreds of thousands of users within hours. Static review fails to catch obfuscated payloads, delayed activation, and behaviors that only surface at runtime. This project proposes a sandboxed behavioral-diff framework that executes two consecutive versions of the same extension under identical, automated browsing conditions and computes a weighted delta score across runtime traces. The system actively triggers dormant payloads via synthetic DOM interaction, records network traffic, API calls, and storage access, and scores the behavioral change using a Random Forest classifier trained on real supply-chain incident data. The result is an explainable, per-update diff report that is actionable for security teams. The project is fully implementable within one four-month semester by two final-year CSE students.

1 Motivation

The browser extension ecosystem runs on a fundamental trust contract: once a user installs an extension, the browser grants it elevated access to every page they visit. This trust contract is not renewed on each update. An extension that passes a careful initial review automatically receives the same privileges after every subsequent silent update, with no re-consent from the user and no re-review by the store.

December 2024 — Cyberhaven Supply-Chain Attack: A threat actor compromised the developer account of Cyberhaven, a legitimate data-loss prevention extension with 400,000 active users. A malicious update was pushed to the Chrome Web Store and silently distributed to all users within hours. The payload exfiltrated session cookies and authentication tokens from targeted financial portals—entirely bypassing static review, permission checks, and endpoint security tools.

Static code analysis and permission-manifest inspection cannot catch this class of attack. Malicious logic can be obfuscated, condition-gated, or delayed. There is currently no open, systematic, version-aware tool that detects such behavioral divergence between consecutive extension releases.

2 Problem Statement

Given two consecutive CRX releases of the same browser extension—version N (trusted baseline) and version $N + 1$ (candidate update)—the system must decide whether the behavioral change between them constitutes a security threat.

The meaningful signal is *change*: new behaviors that did not exist in version N , weighted by their security sensitivity. Formally, the system computes a behavioral delta vector $\Delta = f(T_{N+1}) - f(T_N)$, where T_N and T_{N+1} are runtime traces of each version under identical conditions, and $f(\cdot)$ is a feature extraction function. A classifier then maps Δ to a risk score, outputting an interpretable diff report explaining which behavioral changes drove the alert.

3 Proposed Methodology

The system is structured as a five-stage analysis pipeline:

1. **Scope & Sandbox Construction:** The analysis targets Chromium-based extensions (.crx). Each version is unpacked and loaded into an isolated Docker container running headless Chromium. All outbound traffic is intercepted by mitmproxy.
2. **Active Browsing & Payload Fuzzing:** To activate condition-gated logic, the framework performs synthetic DOM fuzzing (clicking login buttons, populating dummy credential forms) across top Alexa sites, financial login pages, and webmail portals.
3. **Runtime Trace Collection:** The sandbox records a trace spanning four dimensions: (i) outbound network requests, (ii) DOM access events, (iii) storage/cookie access, and (iv) Chrome API calls.
4. **Feature Extraction & Risk Weighting:** Raw traces are converted to a numerical feature vector (Δ). Features are weighted to manage false positives (e.g., domain reputation delta, DOM access intensity, permission delta, exfiltration pattern matching).
5. **Classifier Training & Dataset Generation:** A Random Forest classifier (with XGBoost as secondary) maps Δ to a risk score. Training data is constructed from historical CRX archives of malicious campaigns (e.g., CRXcavator) and a synthetic weaponization corpus where known malicious behaviors are injected into clean extensions.

4 Novelty & Academic Contribution

- **Update-Aware Threat Formulation:** Focuses on temporal supply-chain threats (behavioral divergence between versions) rather than analyzing an extension in isolation.
- **Active Payload Triggering:** Uses synthetic DOM fuzzing to trigger condition-gated payloads that traditional passive crawling approaches miss.
- **Explainable Output:** Produces a per-update diff report detailing which behavioral features changed and why they contributed to the risk score, providing actionable intelligence rather than an opaque label.

5 Implementation & Division of Work

The project relies on an open-source Python stack and divides naturally into two workstreams:

Student A: Bind Pratap Singh (Infrastructure)

- Docker isolation and Playwright browser automation.
- mitmproxy traffic interception and DOM event instrumentation.
- Trace serialization pipeline and Chrome API call logging.

Student B: Suhani Deepak Agrawat (Data & ML)

- Dataset curation, labelling, feature engineering, and weighting.
- Random Forest / XGBoost training and cross-validation.
- FastAPI backend service and Streamlit visualization dashboard.

Evaluation Metrics

Metric	Target / Method
Detection Rate (TPR)	Confirmed malicious update pairs from 2024 campaigns
False Positive Rate (FPR)	Legitimate updates from top-100 extensions over 6 months
Per-pair Latency	End-to-end analysis time from CRX input to report output
Evasion Resistance	Obfuscated variants of known payloads, delayed-activation
Report Quality	Qualitative review: are flagged diff items actionable and accurate?

6 References

References

- [1] S. Bandhakavi et al., "VEX: Vetting Browser Extensions for Security Vulnerabilities," *USENIX Security*, 2010.
https://www.usenix.org/legacy/event/sec10/tech/full_papers/Bandhakavi.pdf
- [2] A. Laperdrix et al., "Browser Fingerprinting: A Survey," *ACM TWEB*, 2020.
<https://arxiv.org/abs/1905.01051>
- [3] S. Karami et al., "Awakening the Web's Sleeper Agents: Misusing Service Workers for Privacy Leakage," *NDSS*, 2021.
<https://www.ndss-symposium.org/ndss-paper/awakening-the-webs-sleeper-agents-misusing-service-workers-for-privacy-leakage>
- [4] F. Fass et al., "HideNoSeek: Camouflaging Malicious JavaScript in Benign ASTs," *ACM CCS*, 2019.
<https://dl.acm.org/doi/10.1145/3319535.3363196>
- [5] "Did I Vet You Before? Assessing the Chrome Web Store's Defenses Against Malicious Extensions," 2024.
<https://arxiv.org/abs/2406.00374>
- [6] S. Singh et al., "A Study on Malicious Browser Extensions in 2025," *arXiv*, 2025.
<https://arxiv.org/abs/2503.04292>
- [7] "JavaSith: A Client-Side Framework for Analyzing Potentially Malicious Extensions," *arXiv*, 2025.
<https://arxiv.org/abs/2505.21263>
- [8] Kaspersky, "Malicious extensions in the Chrome Web Store," 2023.
<https://kaspersky.co.in/blog/dangerous-chrome-extensions-87-million/25861/>
- [9] G. Stock et al., "How Do You Feel About Your Browser Extension?" *IEEE S&P*, 2023.
<https://ieeexplore.ieee.org/document/10179412>